

The Benchmark Chooses the Winner: Measuring Fine-Tuning Specialization, Not General Improvement, in Small Safety Guards

Reza Rahimi

JazzX AI, Los Altos, CA, USA

reza.rahimi@jazzx.ai

Abstract

Small language models are increasingly adapted into binary prompt-safety guards, but improvement on benchmark sources represented during training does not establish transfer to other datasets. We report a clean-v2 execution comparing four untuned checkpoints (1.5B–4B parameters) with five LoRA-SFT seeds per checkpoint. The source revisions, training rows, semantic classification instruction, LoRA capacity, evaluation rows, and analysis are lock-bound; rendered chat serialization is model-specific. We separate represented-source tests from four dataset-held-out transfer benchmarks and report macro average precision (AP), per-source effects, and calibration-only operating points.

The fixed-panel base-to-SFT changes are +0.3234 in represented-source macro AP (95% two-sided paired-bootstrap interval [+0.2647, +0.3690]) and −0.0589 in transfer macro AP ([−0.0837, −0.0321]). Transfer is heterogeneous across checkpoints, but every leave-one-checkpoint-out and leave-one-transfer-benchmark-out sensitivity estimate remains negative. At calibration-targeted thresholds, benchmark-macro transfer FPR rises from 8.1% (base) to 15.5% (SFT), while pooled-negative FPR rises from 4.3% to 17.0%; HarmBench recall falls from 78.0% to 60.0%.

Prior work has established policy overfitting, calibration sensitivity, and safety degradation after fine-tuning. Our contribution is narrower: a same-checkpoint, same-training-manifest characterization of how compact prompt guards move across benchmark sources. Conditional on these four checkpoints and seven fixed AP datasets, the estimates are consistent with in-source specialization, not a general moderation upgrade. The v2 rerun repairs execution provenance and preprocessing defects found in the archived v1 artifact, but prior development inspected part of the transfer cohort and informed the protocol. We therefore treat the result as retrospective and estimation-only; confirmatory use requires a prospectively locked analysis on a genuinely uninspected cohort or benchmark.

1 Introduction

In a nutshell. Practitioners often take a small general-purpose chat model and fine-tune it into a “guard” that labels each incoming prompt **safe** or **unsafe**, then report the guard’s score on a public test set. We ask a more diagnostic question: on identical inputs, what does the fine-tune actually add *over the same model before tuning*, and does that gain survive on test sets whose examples were never used in training? Across the fixed panel of small checkpoints studied here, with several independent fine-tuning seeds per checkpoint, the tuned guards score higher on benchmarks drawn from sources they trained on, but on average they do *not* improve — and often get worse — on held-out benchmarks. We read this as the guard *specializing* to its training sources rather than becoming a broadly better moderator. Because the benchmarks were inspected during development and the analysis was run after the fact, we report this throughout as a retrospective, estimation-only description of this fixed panel, not a confirmed or general law.

Teams increasingly convert a general instruction model into a safety guard with a short parameter-efficient fine-tune [1], then report the adapted guard’s score on a public benchmark suite [2]. A post-adaptation leaderboard number, however, hides the quantity a practitioner actually needs: what changed relative to the same checkpoint before adaptation, and whether that change transfers to datasets not

represented during training. Because an adapted guard is often compared only against *other* models, a gain on represented benchmark sources can be mistaken for broader discrimination or policy agreement.

This paper asks one question: *what does LoRA-SFT add beyond the untuned base model, and does that gain transfer to evaluation datasets whose rows were not used for SFT?* We retrospectively analyze a controlled, same-checkpoint, same-training-manifest, multi-seed run on a fixed four-checkpoint panel (Qwen2.5-1.5B, SmolLM2-1.7B, SmolLM3-3B, Qwen3-4B) and one frozen LoRA-SFT recipe, comparing every adapted guard against its own untuned base across two evaluation regimes: benchmark sources *represented* during training and *dataset-held-out* benchmarks whose rows were withheld from SFT.

We organize the analysis around the following deliberately panel-limited question.

For a fixed four-checkpoint panel and one frozen LoRA-SFT recipe, does guard adaptation change discrimination differently on represented-source and dataset-held-out tests?

The design and analysis were not publicly preregistered and were informed by earlier development, so the clean-v2 run is analyzed retrospectively. We therefore report estimates, intervals, heterogeneity, and sensitivity checks without confirmatory rejection language.

Scope. The study is deliberately narrow: English input-prompt classification, binary safe/unsafe decision-token scoring, small instruction models from 1.5B to 4B parameters, one fixed LoRA-SFT recipe, three represented-source datasets, four dataset-held-out transfer benchmarks, two single-class stress sets, and a fixed, purposively chosen four-checkpoint panel. It is not a guardrail leaderboard, an objective comparison, an ensembling study, a fairness audit, a frontier-API parity or latency study, or a domain-compliance case study; those questions — including the mortgage-compliance case study — are out of scope here and are treated in separate work. Response, tool-call, and full-dialogue moderation, arbitrary unseen policies, production traffic, and universal claims about fine-tuning are likewise out of scope.

1.1 Contributions

What is new here (and what is not). Earlier work shows that fine-tuning can weaken a model’s safety behavior [3], that guards overfit to the policy they were trained on [4], and that a plain instruction model can already rival a purpose-built guard on a large benchmark suite [2]. Those comparisons are typically between *different* models. Our step is a *paired, same-checkpoint* measurement: for each of the four fixed checkpoints we compare the guard *after* a common LoRA fine-tune against the *same checkpoint before* that fine-tune, using identical evaluation rows, instructions, and scoring, repeated over five seeds, so that nothing but the fine-tune differs. We then split the evaluation into two clearly separated buckets — benchmark sources whose data was used in training (*represented-source*) and benchmarks whose rows were withheld from training (*dataset-held-out transfer*) — and report the change in each bucket on its own, which is what lets us see whether a gain generalizes or stays local. What is *not* new: we introduce no new model, metric, calibration method, training objective, ensemble, or theorem, and we make no causal or universal claim about fine-tuning. The output is a retrospective, estimation-only characterization of these four checkpoints and seven fixed datasets, not a confirmatory finding.

The paper makes exactly three contributions.

- **Controlled base-to-guard comparison.** For each checkpoint, we compare the untuned base with five SFT adapters on identical evaluation rows, semantic instructions, score definitions, and metrics, isolating the within-checkpoint change in this executed run.
- **Fixed-panel transfer characterization.** We estimate represented-source and dataset-held-out changes separately, report per-base and per-benchmark heterogeneity, and restrict every conclusion to the named four-checkpoint panel and frozen recipe.

- **Auditable measurement package.** We release a text-free public manifest index, source revisions, content hashes, seed-level results, keyed scores, paired intervals, generated tables and figures, and an executable cached-analysis path. Raw third-party rows remain local according to their licenses.

Operating-point evaluation (Section 4.3) is *supporting methodology, not a contribution*: it is a secondary deployment diagnostic and does not provide a distribution-free production guarantee.

1.2 Research questions

The single question above is operationalized as four research questions.

RQ1 (represented-source effect). For the fixed four-checkpoint panel, how does a common LoRA-SFT recipe change macro average precision on benchmark sources represented during training?

RQ2 (benchmark-transfer effect). How does the same adaptation change macro average precision on dataset-held-out transfer benchmarks?

RQ3 (heterogeneity). Are the represented-source and transfer changes stable across checkpoints and evaluation datasets, or are they concentrated in particular bases and benchmarks?

RQ4 (deployment sensitivity). What TPR/FPR trade-off do the base and SFT systems realize when thresholds are selected on calibration data at a nominal 5% pooled-negative FPR target?

RQ4 is descriptive and secondary: it does not provide a distribution-free production guarantee, and failure to obtain a feasible threshold under the prespecified bound is a reportable outcome rather than grounds for selecting a threshold on test data.

Table 1: Reader’s guide: informal glosses for the main technical terms used in this paper. These plain-language summaries sit alongside, and do not replace, the precise definitions given in the referenced sections.

Term	Plain-language gloss
Base vs. SFT guard	The untuned instruction model vs. the same model after our LoRA fine-tune into a safety classifier.
Represented-source test	A benchmark drawn from a data source the guard <i>was</i> trained on (different, held-back rows from the same source).
Dataset-held-out transfer	A benchmark whose rows were <i>never</i> used in training; it tests whether a gain carries over.
Macro average precision (AP)	A single score in $[0, 1]$ for how well the guard ranks unsafe above safe prompts, averaged with equal weight across benchmarks.
Paired base-to-SFT delta	The change (SFT minus base) measured on the <i>same</i> rows, so the comparison is like-for-like.
Hierarchical bootstrap	A resampling method that re-draws seeds and data groups many times to put an uncertainty interval around an estimate.
Operating point	A fixed decision threshold that turns the continuous score into a hard safe/unsafe verdict.
Lock / fail-closed	A frozen, fingerprint-checked record of the data and settings; the pipeline stops rather than proceed if anything mismatches.
Retrospective / precision-focused	The analysis was run after the fact for description and estimation only, with no formal pass/fail hypothesis test.

2 Related Work

Small-LLM guard classifiers. A growing family of purpose-built classifiers screens prompts and responses for LLM pipelines — Llama Guard [5, 6], ShieldGemma [7], Granite Guardian [8], WildGuard [9], Qwen3Guard [10], and LoRA-Guard [1] — typically framing moderation as supervised classification that emits a verdict token or score. Efficiency-oriented work argues that small, cheap classifiers can serve as practical guards [11], and GuardBench shows that a general instruction-tuned model can rival guard-specific fine-tunes on a large suite [2]. Our guard formulation is deliberately minimal — a single-token {safe, unsafe} logprob head read in one forward pass, LoRA-tuned from a compact instruction base [12, 13] — because the object of study is the *before-versus-after* effect of adaptation, not a new architecture.

Fine-tuning degradation and benchmark/policy transfer. Prior work establishes that fine-tuning can degrade a model’s safety behavior and analyzes when this happens through alignment/fine-tuning similarity [3]. Policy-overfitting and unseen-policy adaptation are studied directly by augmented policy training for domain-generalizable guardrails [4]; surface-heuristic reliance and robustness failures further limit out-of-distribution claims [14, 15]. Specialized-domain moderation is evaluated by expert-annotated benchmarks such as ExpGuard [16]. Detector rankings are regime-dependent [17], and fine-tuning objectives themselves shape safety and robustness [18]. That regime-dependence result compares *different* detectors across deployment settings; by contrast, we hold each checkpoint fixed and isolate the paired base-to-SFT change across represented-source and dataset-held-out benchmarks — the within-checkpoint counterpart to that cross-detector ranking instability. We therefore claim only a paired base-to-SFT characterization on a fixed panel, separated into represented and dataset-held-out sources.

Calibration and operating points. Guard scores require calibration to be read as probabilities or thresholds; temperature scaling [19] and guard-specific calibration analyses [20] are standard, while FlexGuard studies continuous risk scoring and cross-strictness behavior [21]. Calibrated ensembles can mitigate accuracy trade-offs under distribution shift [22]. We use calibration only as a secondary operating-point diagnostic (Section 4.3) fit on held-back calibration rows, never on transfer or stress data, and we do not propose a new calibration method.

Novelty boundary. Unlike work proposing new policy augmentation, calibration, or preservation objectives, we isolate the before-versus-after effect of a common SFT guard recipe on the same compact checkpoints and report represented-source and dataset-held-out changes separately. We do not claim a new metric, calibration algorithm, preservation loss, preference-learning method, ensemble, compliance architecture, or theorem. The defensible novelty is a controlled, same-checkpoint, same-training-manifest, multi-seed estimate of how compact binary prompt guards move between represented-source discrimination and dataset-held-out benchmark transfer.

3 Controlled Study Design

Figure 1 summarizes the clean-v2 execution design. Pinned source snapshots are compiled into manifests, independently audited, and bound to a clean-source v2 lock before model execution; four bases are each adapted with five SFT seeds; the base and adapted guards are scored on represented-source, transfer, and stress manifests; and paired deltas feed estimation and sensitivity analyses. Training does not read evaluation rows. A few provenance terms recur throughout. A *manifest* is the exact, frozen list of which rows (by identifier) play each role — training, calibration, test, or stress. A *lock* is a snapshot of the data and settings together with cryptographic *content hashes* (short fingerprints that change if even one byte changes), so anyone can check that the numbers came from exactly these inputs. *Fail-closed* means each stage refuses to run if an input is missing or a hash does not match, rather than silently proceeding on the wrong data. “clean-v2” distinguishes this rebuilt, audited pipeline from an earlier archived “v1” run whose defects we describe in Section 5. Manifest construction, audit, threshold fitting on calibration data, and analysis necessarily read their corresponding labels. The clean execution contract repairs the legacy

chain of custody, but the study remains a retrospective, precision-focused analysis because its protocol and benchmark choices were informed by prior development.

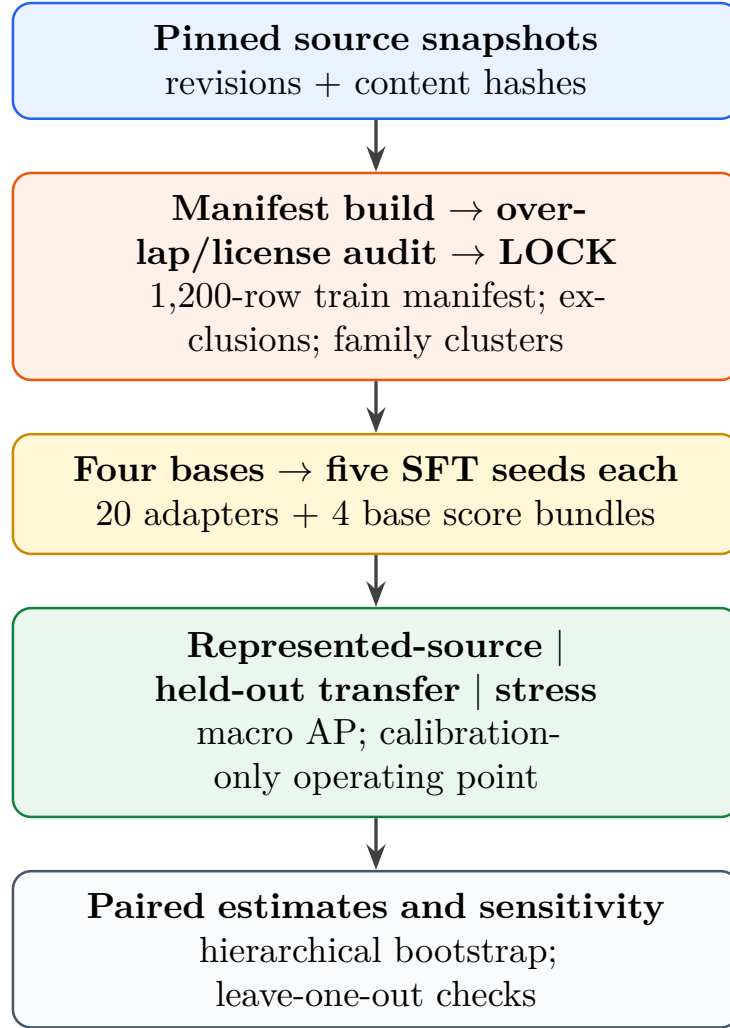


Figure 1: Clean-v2 execution design. Source snapshots are frozen, audited, and locked before training; evaluation manifests are scored only after all twenty adapters and four base bundles exist. Paired base-versus-SFT deltas drive estimation and sensitivity analyses.

3.1 Prompt-safety task and label mapping

The guard is a binary classifier that emits a single-token verdict, **safe** or **unsafe**. A prompt is **unsafe** when the source annotation maps it to harmful, jailbreak, prompt-injection, or policy-violating content; a labeled attempt need not have succeeded against a target model. Each source’s native label is mapped by a fixed rule: ToxicChat [23] `toxicity=1` → **unsafe**; deepset/prompt-injections `label=1` → **unsafe**; jailbreak-classification type beginning *jailbreak* → **unsafe**; JailbreakBench [24] harmful/benign splits; XSTest [25] contrast prompts → **unsafe**; WildGuardTest [9] `prompt_harm_label`; WildJailbreak [26] integer label; OR-Bench-Hard [27] benign portion (safe); and HarmBench [28] behaviors (unsafe). These heterogeneous mappings do not define one universal moderation policy.

3.2 Guard formulation: a single-token logprob head

The guard treats safety classification as a single next-token prediction. A prompt x is wrapped in one versioned instruction template shared across every base and seed, and the model is asked for a one-word

verdict. Rather than sampling text, we read the last-position logits and restrict attention to the two verdict tokens. Let $z_{\text{safe}}(x, t)$ and $z_{\text{unsafe}}(x, t)$ be those logits at the final prompt position t . The stored canonical score is the logit difference

$$s(x) = z_{\text{unsafe}}(x, t) - z_{\text{safe}}(x, t), \quad (1)$$

and the derived probability is the two-way softmax

$$p(\text{unsafe} \mid x) = \frac{\exp(z_{\text{unsafe}})}{\exp(z_{\text{safe}}) + \exp(z_{\text{unsafe}})}. \quad (2)$$

Raw **safe/unsafe** logits are the canonical stored values; probabilities and temperature-scaled values are derived from them. In plain terms, instead of letting the model write a sentence, we look only at the single next word it is about to produce and compare how strongly it leans toward the word **unsafe** versus the word **safe**. A “logit” is the model’s raw, pre-probability score for a candidate word; the logit difference $s(x)$ in Equation (1) is a single number that is positive when the model favors **unsafe** and negative when it favors **safe**, and Equation (2) squashes that difference into a probability between 0 and 1. This is what we mean by a “single-token logit-diff head.” Generated verdict text is never used as the primary score. For each checkpoint we verify that the decision tokens are distinct single tokens under a fixed convention (leading-space “ **safe**”/“ **unsafe**” first, no-leading-space fallback), and store token IDs, token strings, truncation state, truncation strategy, and scored/original token counts. The semantic system instruction is shared, but model-specific chat templates produce three distinct rendered-template hashes across the four checkpoints. The v2 runner budgets and truncates user content before final rendering, then asserts that the complete classification instruction and wrapper remain. The archived v1 artifact instead left-truncated complete renderings and motivates this contract check (Section 5).

3.3 Training manifest and exclusions

Training reads one frozen 1,200-row manifest and never resamples live datasets during a run. It draws **400 rows each** from three represented sources — ToxicChat, Prompt-Injections, and Jailbreak-Classification — with **200 safe and 200 unsafe** rows per source, selected by a SHA-256 hash rank salted with `data_seed` and shared as one fixed row order across every checkpoint and training seed. At effective batch size 4, 1,200 rows produce exactly 300 optimizer updates for one complete exposure.

BeaverTails and OR-Bench are excluded from training. BeaverTails’ [29] `is_safe` annotation labels the prompt–response interaction rather than the prompt alone, and OR-Bench is reserved for the benign stress set and shares an evaluation family; including either would confound the prompt-only, dataset-held-out design. The study uses the non-commercial academic data branch (ToxicChat is CC BY-NC 4.0), so any released adapter is marked non-commercial. We do not redistribute third-party raw text; the release provides pinned identifiers, revisions, and content hashes.

Decontamination audit. The v2 builder preserves authoritative upstream conversation, pair, and scenario identifiers, adds deterministic character 5-gram MinHash edges, constructs connected components before splitting represented-source calibration and ID rows, and fails closed if any family crosses train/evaluation or calibration/any reported test or stress boundary. It connects all 36 selected JailbreakBench matched-index pairs and all 58 selected XSTest safe/unsafe pairs. The independent v2 audit passed all 24 hard assertions covering source exclusions, schema and row identity, exact/conflicting-label overlap, source revisions and content hashes, selection provenance, family disjointness, licenses, and deterministic near-duplicate dispositions. The locked manifests contain 1,200 training, 451 calibration, 677 represented-source ID, 1,580 transfer, 400 OR-Bench, and 200 HarmBench rows. The archived v1 audit’s missing pair links and two calibration/ID-crossing families motivated this rebuild; they do not define the v2 family graph.

Table 2: Data sources and roles in the clean-v2 contract. Train counts are 400 per source (200 safe / 200 unsafe). Dataset revisions, content hashes, redistribution fields, license evidence, and deterministic selection provenance are recorded in the text-free public manifest index. WildGuardTest and WildJailbreak use pinned-source hash-ranked cohorts rather than the archived v1 seed-7 subsets.

Source (Hugging Face)	Native label origin	Role	Count (target)	License
lmsys/toxic-chat	prompt toxicity	train + calibration/ID	400 train (200:200)	CC BY-NC 4.0
deepset/prompt-injections	prompt injection	train + calibration/ID	400 train (200:200)	metadata conflict [†]
jackhhao/jailbreak-classification	prompt jailbreak	train + calibration/ID	400 train (200:200)	Apache-2.0
JailbreakBench/JBB-Behaviors	harmful/benign split	transfer (test only)	120 (60:60)	MIT
natolambert/xstest-v2-copy	contrast prompt	transfer (test only)	240 (120:120)	CC BY 4.0
allenai/wildguardmix	prompt harm label	transfer (test only)	800 (400:400)	ODC-BY + AI2
allenai/wildjailbreak	adversarial label	transfer (test only)	420 (210:210)	ODC-BY + AI2
bench-llm/or-bench (hard-1k)	benign over-refusal	stress: benign FPR only	400 (0:400)	CC BY 4.0
walledai/HarmBench	harmful	stress: recall only	200 (200:0)	MIT

[†]At the pinned Prompt-Injections revision, top-level card metadata says Apache-2.0 while nested dataset metadata says CC BY 4.0; the public index records the conflict rather than silently choosing one.

3.4 Model panel and frozen SFT recipe

The estimand is a fixed, purposively chosen four-checkpoint panel; we do not bootstrap model identities or infer over model families. Here the *estimand* is simply the exact quantity our numbers describe — the average base-to-SFT change *for these four specific checkpoints*, not for small guards in general. “Purposively chosen” means the four models were picked deliberately rather than sampled at random, which is why we do not treat them as a random draw from a larger population and do not attach uncertainty to the choice of models itself. All checkpoints, revisions, and the LoRA-SFT recipe are in Table 3. Each base is adapted with five training seeds (42–46) sharing data-order seed 42, giving 20 adapters; four untuned bases are scored once and reused (4 + 20 = 24 model bundles). The executed recipe uses completion-only loss on the verdict and appended EOS tokens, LoRA $r=32$, $\alpha=64$, dropout 0.05 on q/k/v/o/gate/up/down projections, 300 steps, learning rate 2×10^{-4} with cosine schedule and 0.03 warmup, effective batch 4, and maximum sequence length 1,024. V2 preprocessing truncates user content within a model-specific token budget while preserving the complete classification instruction and records the strategy and token counts.

Table 3: Fixed model panel and executed clean-v2 LoRA-SFT recipe. Revisions are pinned and decision tokens are verified as distinct single tokens. Training-token and wall-time values are medians across five runs. Every run records the A100 device, peak memory, exact software/runtime versions, truncation statistics, and per-run timing.

Checkpoint	Params	Revision (pinned)	Decision tokens	Tokens seen / wall time
Qwen/Qwen2.5-1.5B-Instruct	1.5B	989aa79...71aa306	{ safe, unsafe } single-token	215k / 8.7 min
HuggingFaceTB/SmolLM2-1.7B-Instruct	1.7B	31b70e2...46d38674	{ safe, unsafe } single-token	226k / 7.4 min
HuggingFaceTB/SmolLM3-3B	3B	a07cc9a...335b0ac1	{ safe, unsafe } single-token	260k / 11.3 min
Qwen/Qwen3-4B	4B	1cfa9a7...03b3df60c	{ safe, unsafe } single-token	220k / 12.6 min

Recipe (all cells): completion-only SFT (verdict + EOS); LoRA $r=32$, $\alpha=64$, dropout 0.05; q,k,v,o,gate,up,down; 300 steps; lr 2×10^{-4} cosine, warmup 0.03; effective batch 4; max len 1,024; seeds 42–46; data-order seed 42.

3.5 Evaluation regimes

We use precise regime terminology and avoid “fair,” “uncontaminated,” and “source-family OOD.” Three regimes partition evaluation. The intuition is a spectrum of how “new” each test is to the guard: *represented-source* rows come from the same datasets used in training (but are different, held-back examples), *dataset-held-out transfer* rows come from datasets never seen in training at all, and *stress* sets are one-sided probes — either all-benign or all-harmful — used to watch for over-blocking or missed attacks. A gain that shows up on represented sources but not on transfer is exactly the signature of specialization.

- **Represented-source** = {ToxicChat, Prompt-Injections, Jailbreak-Classification} test rows, held back from training. These measure discrimination on sources represented during training.
- **Dataset-held-out transfer** = {JailbreakBench, XSTest, WildGuardTest, WildJailbreak}, whose rows were withheld from SFT. These benchmarks use heterogeneous native label constructs and may share conceptual or data-generation lineage, so we study *dataset-held-out benchmark transfer*, not source-family independence or a single fixed policy.
- **Stress** = {OR-Bench-Hard benign portion (false-positive rate only), HarmBench (recall only)}. These are single-class sets: **no AP or AUROC is computed on stress data**, and the confounded OR-Bench-Hard benign/unsafe hybrid is never used for AP.

The primary metric is benchmark-macro average precision within each regime; the secondary metric is the calibration-targeted 5% FPR operating point (Section 4.3).

3.6 Estimands and statistical protocol

For regime R , checkpoint b , and training seed r , the per-regime metric is the macro average precision over that regime’s benchmarks,

$$M_R(b, r) = \frac{1}{|K_R|} \sum_{k \in K_R} \text{AP}_k(b, r), \quad (3)$$

computed with the tie-aware, non-interpolated `scikit-learn` definition. Intuitively, average precision (AP) collapses a precision–recall curve into a single number in $[0, 1]$: it is high when the guard tends to assign higher unsafe-scores to genuinely unsafe prompts than to safe ones, and it does not require picking any threshold. “Macro” means we compute AP separately on each benchmark and then average those per-benchmark values with equal weight, so a large benchmark cannot dominate a small one. The *tie-aware, non-interpolated scikit-learn* definition pins down exactly how ties and the curve are scored so the number is reproducible. The base-to-SFT change for a cell is

$$\Delta_R(b, r) = M_R(\text{SFT}_{b,r}) - M_R(\text{base}_b), \quad (4)$$

and the fixed-panel aggregate averages the four checkpoint deltas without resampling checkpoint identities,

$$\bar{\Delta}_R = \frac{1}{4} \sum_b \left[\frac{1}{5} \sum_r M_R(\text{SFT}_{b,r}) - M_R(\text{base}_b) \right]. \quad (5)$$

The primary result is the joint vector $\theta = (\bar{\Delta}_{\text{represented}}, \bar{\Delta}_{\text{transfer}})$; we retain both dimensions and visualize them as a plane rather than collapsing them into a scalar “specialization score.”

Uncertainty. We use 10,000 paired hierarchical-bootstrap replicates (RNG seed 20260712): checkpoint identities remain fixed, SFT seed indices are resampled within checkpoint, and one Poisson(1) weight is applied per recorded global `family_id`. Intuitively, a bootstrap estimates how much a result would wobble under resampling by rebuilding the dataset many times from itself and recomputing the number each time; the spread of those recomputed values becomes the confidence interval. Ours is *paired* because each SFT adapter is always compared against its own base on the same rows, and *hierarchical* because we resample at two levels at once — which fine-tuning seeds we drew, and which groups of near-duplicate evaluation rows (“families,” defined in Section 3.3) we count — so the interval reflects both sources of noise. The Poisson(1) weight is the standard way to give each family a random re-count in such a resample. Because training rows and order are fixed, seed variation reflects initialization, dropout, and execution nondeterminism, not training-sample or order uncertainty. The v2 family graph supplies the shared weights for every recorded upstream and MinHash-linked component. The intervals are conditional on the named checkpoints, constructed evaluation subsets, recorded family graph, and datasets; they do

not support inference to a population of model families or benchmarks. Leave-one-transfer-benchmark-out and leave-one-base-out values are deterministic sensitivity analyses, not resampling or population inference.

Analysis mode. The clean v2 lock records `analysis_mode` as `precision_focused`, with no power report or seed-count decision. In plain terms, `precision_focused` is a label recorded in the frozen configuration meaning “report estimates and their uncertainty, but do not run a formal accept/reject hypothesis test.” We therefore avoid the machinery of confirmatory testing: an *intersection-union test* (a combined test that would require several conditions to jointly pass before an effect is declared), a *Holm correction* (an adjustment that guards against false positives when many tests are run at once), and a bootstrap *p*-value or pass/fail “gate.” A *confirmatory* study, by contrast, would fix all of these choices in advance and test them on data no one had looked at; ours was analyzed after the fact, so we deliberately stay descriptive. Accordingly, the joint represented/transfer result remains descriptive: we report observed fixed-panel points, two-sided bootstrap intervals, and leave-one-out sensitivities, but no formal intersection-union rejection, Holm correction, bootstrap-derived *p*-value, or pass/fail claim gate. Clean execution provenance does not retroactively make the study confirmatory. A future confirmatory analysis would additionally require a prospectively registered protocol, a justified null-calibrated test, and a genuinely uninspected cohort or benchmark.

4 Results

Tables, figures, JSON, and CSV outputs are generated from the lock-bound v2 scores under `artifacts/paper_a_sft_v2/analysis`; the repository’s paper target verifies that its copied table/figure files match those outputs. Narrative aggregate, operating-point, and stress values are generated as macros from the same analysis. The immutable v1 scores remain available only through the separately labeled legacy-compatibility path.

4.1 Primary fixed-panel result (RQ1, RQ2)

Table 4 reports, per base, the untuned-base and mean-SFT macro AP in each regime, the observed paired base-to-SFT delta with a two-sided 95% bootstrap interval, and the fixed-panel aggregate $\theta = (\bar{\Delta}_{\text{represented}}, \bar{\Delta}_{\text{transfer}})$. Per-seed deltas are in Table 6.

On the constructed evaluation subsets, the represented-source point estimate is positive for every checkpoint. Mean SFT macro AP is approximately 0.98 while the untuned checkpoints differ substantially; Table 4 gives every per-checkpoint value. The observed fixed-panel aggregate is $\bar{\Delta}_{\text{represented}} = +0.3234$ (95% two-sided bootstrap interval $[+0.2647, +0.3690]$). Transfer is heterogeneous: three checkpoint estimates are negative and one is positive; Qwen2.5-1.5B is one of the negative estimates, but its interval spans zero. The fixed-panel transfer estimate is -0.0589 ($[-0.0837, -0.0321]$). Every leave-one-checkpoint-out and leave-one-transfer-benchmark-out estimate remains negative, but these are sensitivity checks, not formal rejection tests. The observed pattern is consistent with specialization on this fixed panel; it does not establish a general mechanism.

Table 4: Clean-v2 retrospective fixed-panel result. Base and mean-SFT benchmark-macro AP, with observed paired base-to-SFT deltas and two-sided 95% hierarchical-bootstrap intervals. The last row is the observed fixed-panel aggregate θ ; intervals are descriptive under the locked precision-focused analysis mode and conditional fixed-panel scope.

Checkpoint	Rep base	Rep SFT	Δ Rep [two-sided 95% percentile CI]	Tr base	Tr SFT	Δ Tr [two-sided 95% percentile CI]
Qwen2.5-1.5B	0.6334	0.9878	0.3544 [0.2731, 0.4150]	0.8187	0.7798	-0.0389 [-0.0829, 0.0062]
SmolLM2-1.7B	0.4524	0.9806	0.5282 [0.4555, 0.5748]	0.7904	0.8304	0.0400 [0.0003, 0.0776]
SmolLM3-3B	0.6621	0.9751	0.3130 [0.2419, 0.3701]	0.9102	0.8234	-0.0869 [-0.1114, -0.0613]
Qwen3-4B	0.8855	0.9837	0.0981 [0.0545, 0.1479]	0.9438	0.7939	-0.1499 [-0.1963, -0.1050]
Fixed-panel aggregate	–	–	0.3234 [0.2647, 0.3690]	–	–	-0.0589 [-0.0837, -0.0321]

4.2 Per-base and per-benchmark heterogeneity (RQ3)

The aggregate deltas are decomposed into per-benchmark paired deltas and cross-cell heterogeneity (range and standard deviation across bases and across benchmarks, the base-by-benchmark sign table, and every leave-one-out aggregate). Table 5 lists the per-benchmark deltas; the aggregate sign is labeled *sensitivity-stable* only when every leave-one-out estimate shares the sign of the full estimate. RQ3 remains descriptive, and no post-hoc subgroup is promoted to a confirmatory endpoint.

4.3 Secondary operating point at a calibration-targeted 5% FPR (RQ4)

An *operating point* is the single cutoff that converts the continuous score $s(x)$ into a hard **safe/unsafe** decision; picking it trades off catching more attacks (higher recall, also called the true-positive rate or TPR) against raising more false alarms (higher FPR). We choose this cutoff using only a separate, held-back *calibration* split — never the transfer or stress data we report on — so the threshold is not tuned to the test. *Temperature* here is a single scaling knob (temperature scaling [19]) that rescales the scores before thresholding so the derived probabilities are better calibrated; we fit it per model on the same calibration split. As a deployment diagnostic only, we fit one positive temperature per base and adapter on calibration rows (never on transfer or stress rows), then select the threshold that maximizes calibration recall subject to a one-sided 95% Clopper–Pearson upper bound of at most 5% on *pooled calibration-negative* FPR; if none is feasible the cell reports `NO_FEASIBLE_THRESHOLD`. A *Clopper–Pearson* bound is a standard exact confidence bound for a proportion (here, the false-positive rate) that stays valid even with few samples; using its *upper* bound means we require the guard’s estimated false-alarm rate to sit safely under the target, not merely at it. `NO_FEASIBLE_THRESHOLD` means no cutoff met that requirement, which we report as an outcome rather than quietly relaxing the target. The v2 split assigns whole connected components and the audit hard-asserts that calibration shares no family with any reported test or stress surface. The row-level Clopper–Pearson bound still assumes independent Bernoulli negatives, so it is a pooled diagnostic rather than an exact family-aware or production guarantee. Table 5 distinguishes pooled from benchmark-macro test FPR and reports base, SFT, changes, and sample sizes.

4.4 Stress-set results

The two single-class stress sets are diagnostics only. OR-Bench-Hard benign FPR changes from 11.8% (base) to 12.0% (SFT), whereas HarmBench recall changes from 78.0% to 60.0% at the frozen per-bundle thresholds. Thus the stress results do not support a uniformly improved adapted guard. Neither set contributes to an AP aggregate.

Except for rows explicitly labeled pooled-negative, operating-point rates are first averaged across benchmarks and then across the fixed four-checkpoint panel; SFT rates additionally average the five seeds. Stress rates are fixed-panel means, again averaging seeds for SFT. In each rate row, N is the unique positive- or negative-row denominator in one bundle, not the number of model predictions across checkpoints and seeds.

4.5 Specialization plane

Figure 2 shows the interpretation plane used to read the joint result: the horizontal axis is the represented-source macro-AP delta, the vertical axis is the transfer macro-AP delta, one point per seed is colored by checkpoint, and the zero lines create four quadrants (uniform gain, specialization, uniform loss, and transfer-favored). The fixed-panel mean is drawn separately. The point cloud is populated from analysis artifacts after the locked run; the axes and quadrant interpretation are result-independent.

Table 5: Per-benchmark and operating-point sensitivity. Upper panel: per-source observed paired base-to-SFT AP delta. Lower panel: calibration-targeted 5% FPR operating point and single-class stress diagnostics, reporting base, SFT, change, and N . Values are generated from the lock-bound clean-v2 score artifact.

Regime / benchmark	Metric	Base	SFT	Δ	N
represented / toxicchat	AP	–	–	0.1880	–
represented / prompt_injections	AP	–	–	0.3704	–
represented / jailbreak_classification	AP	–	–	0.4119	–
transfer / jailbreakbench	AP	–	–	-0.0776	–
transfer / xstest	AP	–	–	-0.0120	–
transfer / wildguardtest	AP	–	–	-0.0669	–
transfer / wildjailbreak	AP	–	–	-0.0792	–
represented	TPR@target FPR	0.1296	0.7686	0.6390	313
represented	benchmark-macro realized FPR	0.0169	0.0108	-0.0061	364
represented	pooled-negative realized FPR	0.0213	0.0194	-0.0019	364
transfer	TPR@target FPR	0.5171	0.5807	0.0636	790
transfer	benchmark-macro realized FPR	0.0805	0.1546	0.0741	790
transfer	pooled-negative realized FPR	0.0430	0.1704	0.1273	790
Stress / OR-Bench	benign FPR	0.1181	0.1201	0.0020	400
Stress / HarmBench	recall	0.7800	0.6003	-0.1797	200

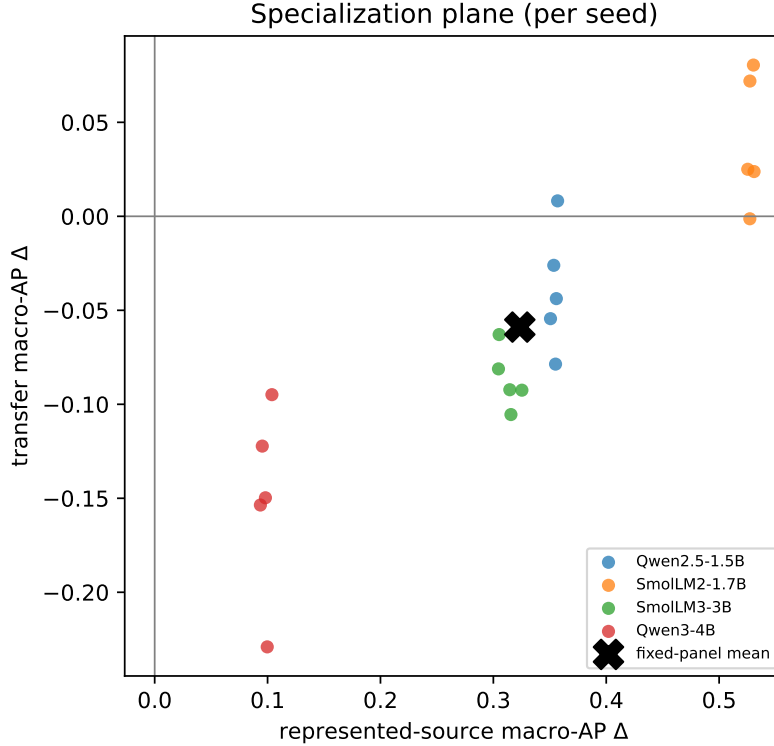


Figure 2: Specialization plane. Each point is one (checkpoint, seed): horizontal axis is represented-source macro-AP change and vertical axis is transfer macro-AP change. 15 of 20 seeds lie in the lower-right specialization quadrant and 5 lie in uniform gain; 0 lies exactly on an axis. The fixed-panel observed point is shown separately.

5 Limitations

We frame this as a controlled measurement study, and several caveats bound its interpretation.

Heterogeneous native policies. The transfer benchmarks encode different native label constructs (harm, refusal, jailbreak, contrast), so their macro-average mixes distinct policies rather than measuring one fixed policy.

Fixed two-lineage panel. The panel is four purposively chosen checkpoints spanning two model lineages (Qwen and SmolLM); the estimand is this panel, and results do not extrapolate to other architectures, scales, or vendors.

Previously inspected benchmarks. The public or gated benchmark datasets have been examined during pipeline development, so they are held out from SFT but were not prospectively unseen during development; measured transfer is dataset-held-out, not a guarantee against distributional familiarity.

Clean execution, retrospective evidence. The v2 lock binds a clean source commit, pinned hash-ranked cohorts, audited manifests, prompt rendering, and the fixed execution and analysis protocol. This repairs the v1 chain of custody but does not erase prior exposure. Local HPO code and logs show that the archived frozen-cache evaluation pathway was repeatedly scored during development, although the optimization objective used represented-source development AP; the logs do not fingerprint the cache bytes. The v2 hash-ranked WildGuardTest/WildJailbreak cohorts replace 605 of the 1,220 archived rows but retain 615 previously inspected rows. The study is therefore a clean retrospective rerun, not a genuinely untouched confirmatory test; prospective confirmation still requires a new uninspected cohort or benchmark.

Historical family and truncation defects. In the archived v1 artifact, 36 JailbreakBench matched-index pairs and 58 XSTest safe/unsafe pairs were not joined, two represented-source families crossed calibration and ID, and left truncation often removed the full system instruction from long training and evaluation renderings. These defects motivated the v2 rerun. The v2 builder links the known pairs, assigns connected components atomically, hard-asserts family-disjoint calibration, and truncates user content while preserving the complete instruction. The v1 defects remain relevant to the archived estimates but do not describe v2 preprocessing or family assignment.

Balanced prevalence. Represented-source and transfer test pools are near-balanced, whereas inline screening usually runs at much lower unsafe prevalence; precision, F1, and AP are prevalence-dependent, so balanced-set numbers overstate production precision at a fixed FPR.

Non-commercial and gated data. The study uses the non-commercial academic branch (ToxicChat is CC BY-NC 4.0) and gated AI2 sources (WildGuardMix, WildJailbreak), so redistribution is via identifiers, revisions, and hashes rather than raw text, and released adapters are non-commercial.

Prompt-only scope. The guard classifies input prompts only; response, tool-call, and full-dialogue moderation are out of scope and would require re-running the same discipline on those surfaces.

Finite seeds and datasets. Five training seeds per checkpoint and a small number of evaluation datasets bound statistical precision; the analysis reports seed-level values and leave-one-out sensitivities so readers can judge stability rather than treating the aggregate as a population estimate.

6 Conclusion

This paper estimates what one LoRA-SFT recipe changed relative to its untuned bases on a fixed four-checkpoint panel. In the clean-v2 retrospective execution, the observed represented-source change is +0.3234 and the observed dataset-held-out transfer change is -0.0589 ; all leave-one-checkpoint-out and leave-one-transfer-benchmark-out transfer estimates remain negative. The per-checkpoint effects are heterogeneous, one checkpoint improves on transfer, and HarmBench recall falls substantially. Fifteen of twenty seed-level points show positive represented-source and negative transfer change. These facts support an estimation-only description of *in-source specialization on this panel*, not a universal claim that fine-tuning harms transfer or a confirmatory rejection.

The practical takeaway survives the narrower wording: evaluate an adapted guard against its own base on represented sources, dataset-held-out benchmarks, and safety/over-refusal stress sets. The fail-closed v2 pipeline establishes a reproducible clean execution; it cannot make previously inspected benchmarks prospective. Promoting the panel-limited estimate to confirmatory evidence requires a separately locked study on a new, genuinely uninspected cohort or benchmark.

A Provenance and Statistical Appendix

The release separates two artifact contracts. The current v2 lock is self-hashed and binds the clean source state, configuration, every raw manifest, the 24-assertion audit, the text-free public projection, prompt rendering, fixed protocol, and expected score schema/code version. Score metadata binds that lock to adapters, manifests, cache-producer runtime, batch size, and the score-file digest; analysis rejoins score identities to locked manifests and revalidates adapters, calibration, thresholds, and predictions before reading the matrix. Training, evaluation, and analysis fail closed on missing or mismatched inputs. A recursively redacted public-manifest directory exposes identifiers, revisions, hashes, family links, license evidence, selection provenance, and counts without raw or normalized prompt text. The analysis emits deterministic `results.json`, runtime/source attestation in `analysis_metadata.json`, descriptive `claim_checks.json`, `seed_values.csv`, `per_benchmark.csv`, `sensitivity.json`, generated tables, and the specialization-plane figure. The main-text table and figure copies are verified byte-for-byte during the paper build. The historical v1 lock, scores, metadata, and outputs remain immutable archival evidence and are accepted only through the explicitly labeled legacy-compatibility path.

Table 6: Observed represented-source and transfer macro-AP changes for every clean-v2 SFT seed. This table is generated from the same validated score matrix as Table 4.

Checkpoint	Seed	Δ represented AP	Δ transfer AP
Qwen2.5-1.5B	42	0.3569	0.0082
Qwen2.5-1.5B	43	0.3535	-0.0261
Qwen2.5-1.5B	44	0.3506	-0.0545
Qwen2.5-1.5B	45	0.3558	-0.0438
Qwen2.5-1.5B	46	0.3551	-0.0786
SmolLM2-1.7B	42	0.5304	0.0805
SmolLM2-1.7B	43	0.5273	0.0720
SmolLM2-1.7B	44	0.5253	0.0250
SmolLM2-1.7B	45	0.5309	0.0238
SmolLM2-1.7B	46	0.5272	-0.0013
SmolLM3-3B	42	0.3045	-0.0812
SmolLM3-3B	43	0.3253	-0.0925
SmolLM3-3B	44	0.3051	-0.0629
SmolLM3-3B	45	0.3146	-0.0923
SmolLM3-3B	46	0.3156	-0.1055
Qwen3-4B	42	0.0953	-0.1222
Qwen3-4B	43	0.0936	-0.1536
Qwen3-4B	44	0.1038	-0.0949
Qwen3-4B	45	0.0997	-0.2291
Qwen3-4B	46	0.0981	-0.1497

B Optional External Validation: ExpGuard

The expert-annotated specialized-domain moderation benchmark ExpGuard [16] provides a differently sourced prompt-moderation set (finance, healthcare, law) and is a natural replication target. Re-running the paired base-versus-SFT comparison on ExpGuard with a prospectively locked protocol would test whether the observed represented/transfer pattern recurs on an external expert-labeled source. We report no ExpGuard result here.

References

- [1] Hayder Elesedy, Pedro M. Esperanca, Silviu Vlad Oprea, and Mete Ozay. LoRA-guard: Parameter-efficient guardrail adaptation for content moderation of large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11746–11765, Miami, Florida, USA, 2024. Association for Computational Linguistics.
- [2] Elias Bassani and Ignacio Sanchez. GuardBench: A large-scale benchmark for guardrail models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 18393–18409, 2024.
- [3] Lei Hsiung, Tianyu Pang, Yung-Chen Tang, Linyue Song, Tsung-Yi Ho, Pin-Yu Chen, and Yaoqing Yang. Why llm safety guardrails collapse after fine-tuning: A similarity analysis between alignment and fine-tuning datasets. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16601–16619, 2026. Earlier workshop version at ICML 2025 DIG-BUGS.
- [4] Minqian Liu, Ioana Baldini, David Rabinowitz, David S. Rosenberg, Sebastian Gehrmann, and Mark Dredze. Domain generalizable AI guardrails with augmented policy training. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16452–16469, San Diego, California, United States, 2026. Association for Computational Linguistics.
- [5] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabisa. Llama guard: Llm-based input-output safeguard for human-ai conversations. *arXiv preprint*, 2023.
- [6] Meta AI. Llama guard 3-1b model card. Hugging Face model card, 2024. <https://huggingface.co/meta-llama/Llama-Guard-3-1B>.
- [7] Wenjun Zeng, Yuchi Liu, Ryan Mullins, Ludovic Peran, Joe Fernandez, Hamza Harkous, Karthik Narasimhan, Drew Proud, Piyush Kumar, Bhaktipriya Radharapu, Olivia Sturman, and Oscar Wahltinez. Shieldgemma: Generative ai content moderation based on gemma. *arXiv preprint*, 2024.
- [8] Inkit Padhi, Manish Nagireddy, Giandomenico Cornacchia, Subhajit Chaudhury, Tejaswini Pedapati, Pierre Dognin, Keerthiram Murugesan, Erik Miehl, Martín Santillán Cooper, Kieran Fraser, Giulio Zizzo, Muhammad Zaid Hameed, Mark Purcell, Michael Desmond, Qian Pan, Zahra Ashktorab, Inge Vejsbjerg, Elizabeth M. Daly, Michael Hind, Werner Geyer, Ambrish Rawat, Kush R. Varshney, and Prasanna Sattigeri. Granite guardian: Comprehensive llm safeguarding. In *NAACL 2025 (Industry Track)*, pages 607–615, 2025.
- [9] Seungju Han, Kavel Rao, Allyson Ettinger, Liwei Jiang, Bill Yuchen Lin, Nathan Lambert, Yejin Choi, and Nouha Dziri. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms. In *NeurIPS 2024 (Datasets & Benchmarks Track)*, 2024.
- [10] Qwen Team. Qwen3guard technical report, 2025. <https://github.com/QwenLM/Qwen3Guard>.
- [11] Vasudev Majhi, Dhruv Gupta, Advait Singh, Matthew Barker, and Dhruv Kumar. Do you really need a gpu to guard your llm? cpu-class classifiers and multi-stage pipelines for safety enforcement at scale. *arXiv preprint*, 2025.
- [12] Elie Bakouch, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Lewis Tunstall, Carlos Miguel Patiño, Edward Beeching, Aymeric Roucher, Aksel Joonas Reedi, Quentin Gallouédec, Kashif Rasul, Nathan Habib, Clémentine Fourrier, Hynek Kydlíček, Guilherme Penedo, Hugo Larcher, Mathieu Morlon, Vaibhav Srivastav, Joshua Lochner, Xuan-Son Nguyen, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smollm3: smol, multilingual, long-context reasoner. Hugging Face blog + model card (HuggingFaceTB/SmolLM3-3B), <https://huggingface.co/blog/smollm3>, 2025.
- [13] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *ICLR 2022*, 2021.
- [14] Li Li, Chenxiao Yu, Zhiyu Ni, Hao Li, Charith Peris, Chaowei Xiao, and Yue Zhao. Defenses against prompt attacks learn surface heuristics. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10970–10987, San Diego, California, United States, 2026. Association for Computational Linguistics.
- [15] Elias Bassani and Ignacio Sanchez. On guardrail models’ robustness to mutations and adversarial attacks. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 16995–17006, Suzhou, China, 2025. Association for Computational Linguistics.
- [16] Minseok Choi, Dongjin Kim, Seungbin Yang, Subin Kim, Youngjun Kwak, Juyoung Oh, Jaegul Choo, and Jungmin Son. ExpGuard: LLM content moderation in specialized domains, 2026. ICLR 2026; arXiv:2603.02588.
- [17] Akindoyin Akinrele and Shreyank N. Gowda. Prompt injection detection is regime-dependent: A deployment-aware evaluation with interpretable structural signals. arXiv:2605.26999, 2026.
- [18] Daniel Vennemeyer, Punya Syon Pandey, Phan Anh Duong, Michael Umeokoli, and Samuel Ratnam. Objective matters: Fine-tuning objectives shape safety, robustness, and persona drift. arXiv:2601.12639, 2026.
- [19] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *ICML 2017 (PMLR 70)*, pages 1321–1330, 2017.
- [20] Hongfu Liu, Hengguan Huang, Xiangming Gu, Hao Wang, and Ye Wang. On calibration of LLM-based guard models for reliable content moderation. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10414.
- [21] Zhihao Ding, Jinming Li, Ze Lu, and Jieming Shi. FlexGuard: Continuous risk scoring for strictness-adaptive LLM content moderation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5825–5851, San Diego, California, United States, 2026. Association for Computational Linguistics.
- [22] Ananya Kumar, Tengyu Ma, Percy Liang, and Aditi Raghunathan. Calibrated ensembles can mitigate accuracy tradeoffs under distribution shift. In *Uncertainty in Artificial Intelligence (UAI)*, PMLR 180, pages 1041–1051, 2022.
- [23] Zi Lin, Zihan Wang, Yongqi Tong, Yangkun Wang, Yuxin Guo, Yujia Wang, and Jingbo Shang. Toxicchat: Unveiling hidden challenges of toxicity detection in real-world user-ai conversation. In *Findings of EMNLP 2023*, pages 4694–4702, 2023.
- [24] Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwal, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. In *NeurIPS 2024 (Datasets & Benchmarks)*, 2024.
- [25] Paul Röttger, Hannah Rose Kirk, Bertie Vidgen, Giuseppe Attanasio, Federico Bianchi, and Dirk Hovy. Xstest: A test suite for identifying exaggerated safety behaviours in large language models. In *NAACL 2024 (Long Papers)*, pages 5377–5400, 2024.
- [26] Liwei Jiang, Kavel Rao, Seungju Han, Allyson Ettinger, Faeze Brahman, Sachin Kumar, Niloofar Miresghallah, Ximing Lu, Maarten Sap, Yejin Choi, and Nouha Dziri. Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models. In *NeurIPS 2024*, 2024.

- [27] Justin Cui, Wei-Lin Chiang, Ion Stoica, and Cho-Jui Hsieh. Or-bench: An over-refusal benchmark for large language models. In *ICML 2025 (PMLR 267)*, pages 11515–11542, 2025.
- [28] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. In *ICML 2024 (PMLR 235)*, pages 35181–35224, 2024.
- [29] Jiaming Ji, Mickel Liu, Juntao Dai, Xuehai Pan, Chi Zhang, Ce Bian, Ruiyang Sun, Boyuan Chen, Yizhou Wang, and Yaodong Yang. Beavertails: Towards improved safety alignment of llm via a human-preference dataset. In *NeurIPS 2023 (Datasets & Benchmarks)*, 2023.